

TrustResume

Detailed Design

A comprehensive reference for the codebase's architecture, data flow, testing model, and deployment — companion to [architecture/high-level-design.md](#) and [docs/code-walkthrough.md](#).

TrustResume — Detailed Design

A comprehensive reference for understanding this codebase end to end: what it does, how the pieces fit together, how data flows through one generation run, how it's tested, and how it's deployed. This is a companion to (not a replacement for) the shorter docs already in the repo:

- architecture/high-level-design.md — components + what changed from the original
- architecture/decisions/*.md — ADRs (why, not just what)
- docs/code-walkthrough.md — narrative learning guide (written in Chinese)
- CLAUDE.md — commands, conventions, package map

This document goes one level deeper: concrete function signatures, exact control-flow, schema shapes, and the deployment topology.

1. What the system does

TrustResume — System Overview

UI → HTTP → Facade → Orchestrator → Agents → Storage

agents/ — six pure input → output steps, sequenced by the orchestrator

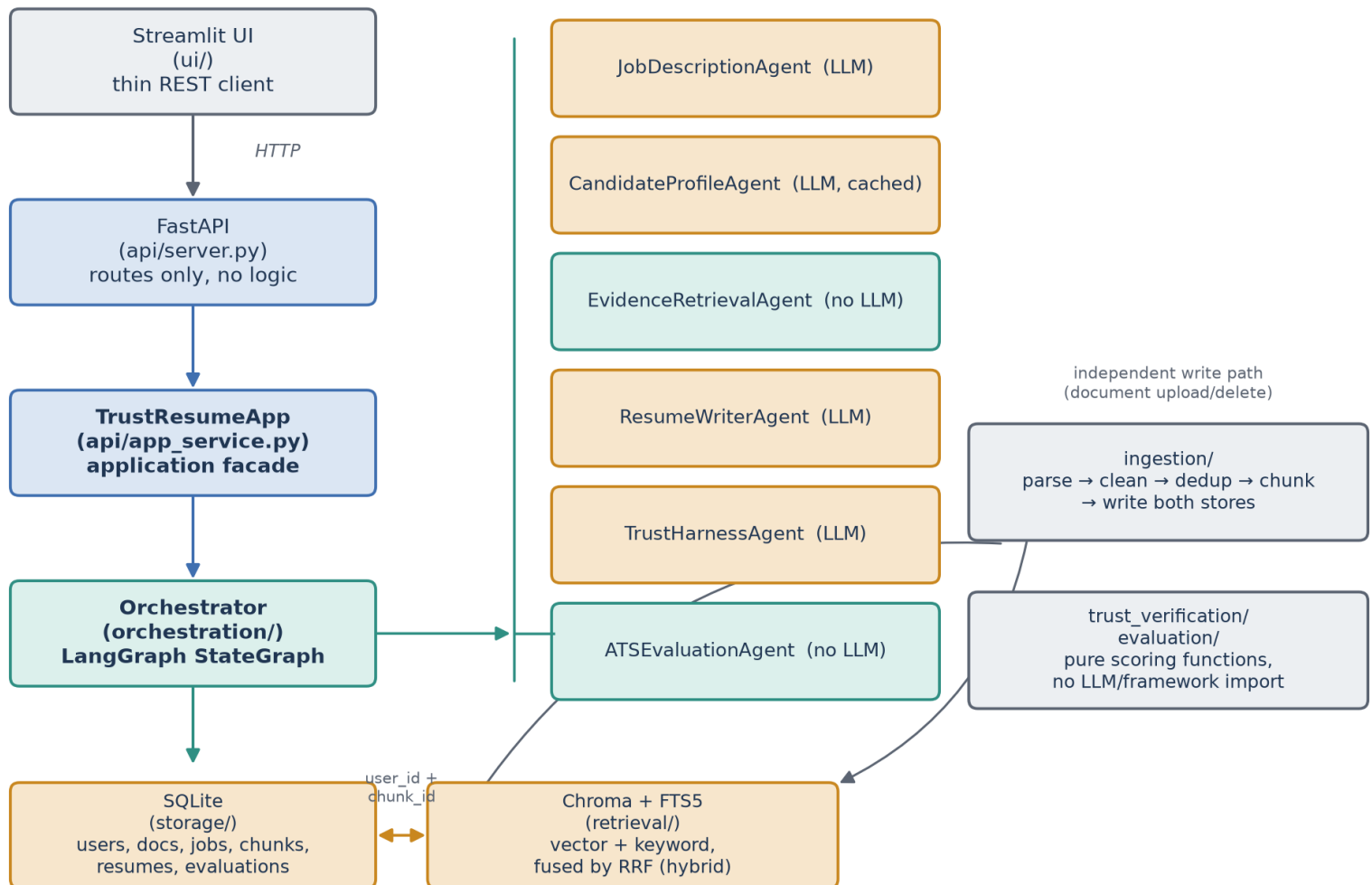


Figure 1. System overview — UI through HTTP to the facade, orchestrator, agents, and the two backing stores.

TrustResume generates an ATS-friendly résumé draft tailored to a job posting, grounded entirely in a candidate's own uploaded evidence (résumés, project reports, STAR stories, certifications) — and, distinctively, **verifies its own output** for hallucinated claims before returning it. The core idea: an AI-generated résumé that lies to beat a keyword filter is worse than an honest one with gaps, so every claim is checked against retrieved evidence and scored (0–100) before anything is shown to the user.

Three techniques compose to do this:

1. **RAG** — candidate documents are chunked, embedded, and stored; a job posting's structured requirements become a query that retrieves the most relevant evidence for that job.
 2. **Multi-agent pipeline** — six single-purpose steps (extract job → resolve candidate profile → retrieve evidence → write draft → verify claims → score ATS coverage), each a pure function from typed input to typed output, sequenced by one orchestrator.
 3. **Quality loop** — after writing, the draft is scored on two independent axes (Trust, ATS); if it fails either threshold, deterministic feedback is built from *why* it failed and the writer rewrites, up to a hard cap.
-

2. Repository layout

```
src/trustresume/
├── models/                Pydantic schemas (extra="forbid"), zero framework deps – everyone
├── imports this
├── storage/               SQLite: users, documents, jobs, chunks (+FTS5), resumes, evaluations,
├── profile cache
├── retrieval/             FastEmbed embeddings + Chroma vector store + hybrid
├── (vector+keyword/RRF) retriever
├── ingestion/             parse (unstructured) → clean → dedup → chunk (LangChain) → write
├── both stores
├── prompting.py           shared prompt-injection defense (wrap_untrusted /
├── UNTRUSTED_INPUT_NOTICE)
├── agents/                six pure input-output agents wrapping
├── chat_model.with_structured_output(Schema)
├── orchestration/         Orchestrator (LangGraph StateGraph), CandidateProfileService,
├── build feedback
├── trust_verification/     Trust Harness prompt/formatting/report-assembly – pure functions,
├── no LLM import
├── evaluation/            ATS keyword-coverage scoring – pure functions
├── export/                render a ResumeDraft to Markdown / PDF (fpdf2)
├── api/                   TrustResumeApp facade, model_factory (provider-agnostic LLM), test
├── provider, FastAPI server, wire schemas
├── ui/                    Streamlit frontend – thin REST client only, no backend imports
├── poc/                   standalone LLM connectivity smoke test (not part of the app;
├── excluded from coverage)
├── logging_config.py       stdlib logging + JSON formatter, configured once at process entry
```

Dependency direction is strictly bottom-up in this list: `models/` is imported by everything and imports nothing project-local; `api/` wires everything else together and is imported by nothing project-local (only `ui/` talks to it, and only over HTTP).

3. Data model (`models/`)

All pydantic `BaseModels` with `model_config = ConfigDict(extra="forbid")` — an unexpected field from an LLM's structured output fails loudly instead of silently passing through. Key types, in the order they appear in a generation:

Model	Produced by	Shape
JobDescription	JobDescriptionAgent	raw_text, title, company, seniority (enum), required_skills/preferred_skills/responsibilities/keywords (all list[str])
CandidateProfile	CandidateProfileAgent (cached)	name, summary, skills, certifications — job-independent
EvidenceChunk / EvidenceSet	retrieval	chunk: chunk_id, user_id, document_id, document_type, source_document, text, score (nullable). Set: user_id, query, chunks: list[EvidenceChunk]
ResumeDraft	ResumeWriterAgent	summary: str, sections: list[ResumeSection] (heading + bullets), iteration: int
TrustReport	TrustHarnessAgent	claims: list[VerifiedClaim], score: float [0,100], iteration. VerifiedClaim: text, category (enum: SKILL/EXPERIENCE/CERTIFICATION/ACHIEVEMENT/OTHER), status (enum: SUPPORTED/PARTIALLY_SUPPORTED/UNSUPPORTED), evidence_chunk_ids, rationale
ATSReport	ATSEvaluationAgent	score, matched_keywords, missing_keywords, notes, iteration
WorkflowState	Orchestrator.run()	the whole run — see §6

Model	Produced by	Shape
QualityGate	config	min_trust_score=90, min_ats_score=85, max_iterations=3

Two properties worth internalizing because they're load-bearing elsewhere:

- **VerifiedClaim.is_hallucination** — UNSUPPORTED *and* category is not OTHER. A stylistic phrase with no evidence isn't a hallucination; an unsupported skill/experience/certification/achievement claim is.
- **TrustReport.compute_score** — SUPPORTED→1.0, PARTIALLY_SUPPORTED→0.5, UNSUPPORTED→0.0, averaged $\times 100$. **An empty claim list scores 0**, not 100 — nothing verifiable is treated as untrustworthy, not neutral. This is why the offline test LLM provider (whose synthesized claims list is always empty) always sits at Trust=0 and never passes the gate.

4. Agents (agents/)

Every agent is a pure input -> output async function: it takes everything it needs as arguments, returns one models/ object, and never calls another agent or touches orchestrator state (agents/base.py's doc comment states this as a hard rule). The orchestrator is the only thing that sequences them.

Agent	LLM?	Signature
JobDescriptionAgent	yes	run(job_posting: str) -> JobDescription
CandidateProfileAgent	yes	run(candidate_text: str) -> CandidateProfile
EvidenceRetrievalAgent	no	run(*, user_id, job, document_ids=None) -> EvidenceSet
ResumeWriterAgent	yes	run(*, job, evidence, feedback=None, iteration=0) -> ResumeDraft
TrustHarnessAgent	yes	run(*, draft, evidence) -> TrustReport

Agent	LLM?	Signature
ATSEvaluationAgent	no	run(*, draft, job) -> ATSReport

Retrieval and ATS scoring are deliberately *not* LLM calls — both are deterministic, so routing them through a model would only add cost and nondeterminism for no benefit.

4.1 The LangChain integration point

Every LLM-backed agent constructs its call the same way, via `agents/base.py`:

```
with_structured_retry(model, Schema) =
model.with_structured_output(Schema).with_retry(stop_after_attempt=3)
```

`with_structured_output` binds a pydantic schema as a tool the model must call; the provider SDK handles the tool-call/parse round trip. Every constructor takes the model directly (`ModelInput = BaseChatModel`) — there is no post-construction `.override(model=...)` hook in LangChain (unlike pydantic-ai), so tests inject a fake model at construction time instead.

`ensure_type(result, ExpectedType)` narrows `ainvoke`'s Any return with a runtime `isinstance` check — deliberately not a bare `assert` (which disappears under python -O), so a future SDK behavior change surfaces as a clear `TypeError` at the call site rather than a confusing `AttributeError` downstream.

4.2 Prompt-injection defense (prompting.py)

Every prompt that interpolates externally-sourced text (a job posting, a candidate document, retrieved evidence) wraps it with `wrap_untrusted(tag, text) → <tag>\n{text}\n</tag>`, paired with `UNTRUSTED_INPUT_NOTICE` in the system prompt telling the model to treat tagged content as data, never instructions. This module is intentionally framework-independent (no `langchain_core` import) so `trust_verification/` — which has a zero-LangChain policy (ADR-0004) — can reuse it without pulling that dependency in.

4.3 ResumeWriterAgent's two-schema trick

The one agent with real complexity beyond "call model, return result." Bedrock Claude in practice emits structural noise that would crash strict `ResumeDraft` validation:

- a section with an empty heading (a stray bullet with nowhere to go),
- the summary emitted as its own "Summary"/"Profile" section instead of the dedicated summary field,
- a bare group-label section with no bullets (e.g. "Professional Experience" followed by one section per employer).

So the agent binds a lenient private schema, `_DraftExtraction` (`heading: str = ""` allowed), and `to_resume_draft()` cleans it up in code: empty headings become "Additional Information"; a leading section whose heading matches {summary, professional summary, profile, objective} gets folded into the summary field instead of kept as a section; bulletless sections are dropped entirely. This is the same "LLM emits, code cleans up" split the Trust Harness uses for scoring — see `test_agents.py::test_resumeAgent_emptySectionHeading_relabeledNotDropped`.

4.4 TrustHarnessAgent's citation sanitization

After classifying claims, `drop_unknown_citations` strips any `evidence_chunk_ids` entry that doesn't correspond to a chunk actually in the `EvidenceSet` passed in — an unchecked citation (hallucinated or copied from elsewhere) would otherwise be indistinguishable from a real, auditable one. This doesn't change a claim's SUPPORTED/PARTIALLY/UNSUPPORTED

status, only its cited evidence ids.

5. Retrieval (retrieval/)

Two things get searched and fused, both scoped by `user_id`:

5.1 ChromaVectorStore (vector)

Wraps `langchain_chroma.Chroma`. Three-method surface: `upsert_chunks`, `search`, `delete_chunks`.

- `chunk_id` is used directly as the Chroma document id (Chroma accepts arbitrary strings — no `uuid5` indirection needed, unlike the original Qdrant-backed version).
- Every `search()` call passes `filter={"user_id": ...}` — this is the **entire** user-isolation mechanism (ADR-0001). There is no filter-less search path in the codebase.
- `document_ids`, when given, adds `$and: [{"user_id": ...}, {"document_id": {"$in": [...]}]}` — job-scoped retrieval. An empty `document_ids` list short-circuits to an empty `EvidenceSet` without calling Chroma (an empty `$in` is itself invalid there).
- Chroma's `similarity_search_with_score` returns cosine **distance** (lower = closer); the store converts `score = 1.0 - distance` so the rest of the app's convention ("`higher EvidenceChunk.score = more relevant`") holds.

5.2 ChunkRepository.search keywords (keyword, BM25)

SQLite FTS5 external-content virtual table chunks `fts` indexes `chunks.text` without duplicating it (kept in sync by AFTER INSERT/UPDATE/DELETE triggers on chunks — see `storage/schema.py`). `_to_fts5_query` tokenizes free text into bare alphanumeric words, quotes each as its own phrase, OR-joins them — sidesteps FTS5's MATCH syntax choking on `-/*/://parens`, which job descriptions reliably contain.

5.3 HybridRetriever (fusion)

`retrieval/hybrid.py`. Queries both sources for `candidates_per_source=20` each (more than the final limit, so fusion has overlap to work with), then fuses by **Reciprocal Rank Fusion**:

$$\text{score}(\text{chunk}) = \sum \text{over lists containing it of } 1 / (k + \text{rank_in_that_list}) \quad (k=60)$$

RRF fuses on **rank position**, never raw score — Chroma's cosine similarity `[0,1]` and SQLite's unbounded `bm25()` aren't on comparable scales, so averaging them would be meaningless. A chunk found by both sources (even at different ranks) outranks one found by only one — the property that makes this genuinely better than "two searches concatenated."

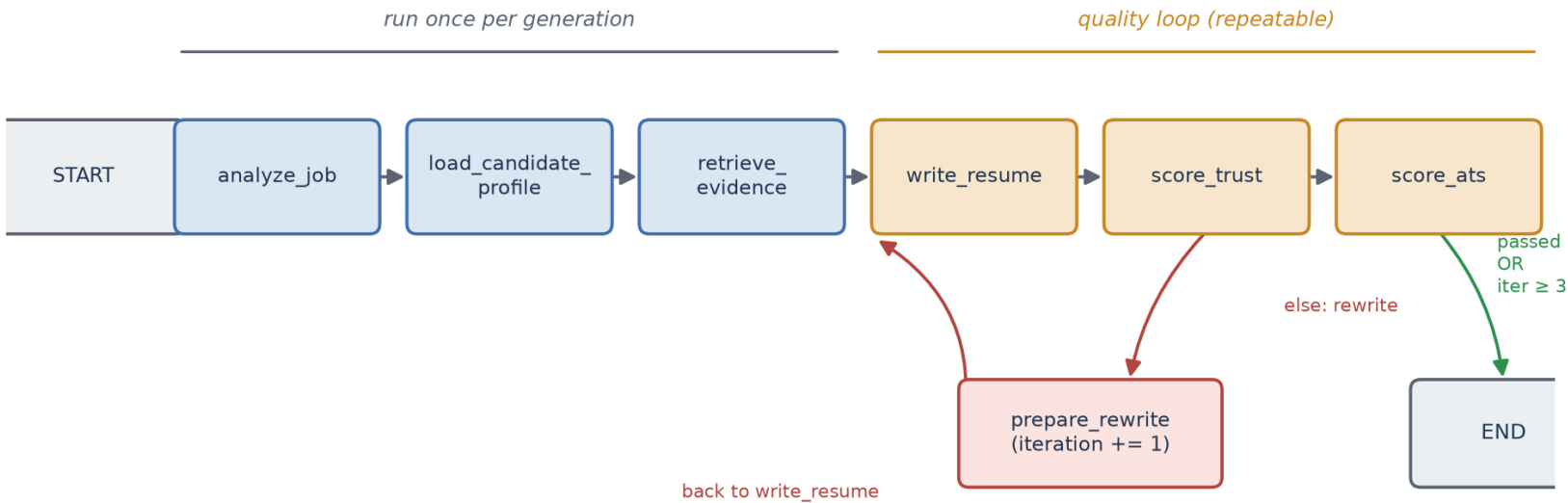
`HybridRetriever.search(user_id, query, limit, document_ids=None)` -> `EvidenceSet` matches `ChromaVectorStore.search`'s exact signature — both satisfy a Protocol (`agents/retrieval_agent.py`'s `_Retriever`), so `EvidenceRetrievalAgent` and `TrustResumeApp.search_evidence` are agnostic to which one they're constructed with.

5.4 Embeddings

`FastEmbedEmbeddings` implements `langchain_core.embeddings.Embeddings` around `fastembed's BAAI/bge-small-en-v1.5` (384-dim), lazy-loaded on first real embed call (constructing a store never triggers a download). This is a small, general-purpose model not tuned on résumé/job vocabulary — its weakness on exact technical terms is *why* hybrid retrieval exists at all (ADR-0010): "AWS Lambda" shouldn't lose to "AWS EC2" on pure semantic similarity when the keyword path can hit it exactly.

6. Orchestration (orchestration/orchestrator.py)

Orchestrator — LangGraph StateGraph (orchestration/orchestrator.py)



`_route()` reads ``iteration`` BEFORE `prepare_rewrite` increments it — default `max_iterations=3` ⇒ iterations 0,1,2,3 all run ⇒ 4 total drafts, not 3.

Figure 2. The LangGraph StateGraph — one-time setup nodes, the repeatable quality loop, and the pre-increment iteration check that yields 4 drafts.

The single owner of control flow, built on a LangGraph StateGraph. Public contract is a plain async method — everything above it (the facade) is unaware it's a graph internally:

```
async def run(*, user_id, job_posting=None, job=None, job_id=None,
document_ids=None, gate=None) -> WorkflowState
```

Exactly one of `job_posting` (raw text, extracted fresh every call — the legacy path) or `job` (a pre-extracted, typically persisted `JobDescription` — extraction is skipped) must be given.

6.1 The graph

```
START → analyze_job → load_candidate_profile → retrieve_evidence
→ write_resume → score_trust → score_ats
```

```
conditional edge: route
├ "end" → END
└ "rewrite" → prepare_rewrite → (back to) write_resume
```

- `analyze_job`, `load_candidate_profile`, `retrieve_evidence` run **once** per generation — the job and the candidate's evidence don't change between rewrites, only the draft does.
- `write_resume` → `score_trust` → `score_ats` is the repeatable quality loop.

Internal state is a private `TypedDict` (_GraphState)`, never exposed outside this module. `drafts/trust_reports/ats_reports` are `Annotated[list[X], operator.add]` — LangGraph's append-only reducer, so every node returning `{"drafts": [draft]}` appends rather than overwrites, preserving the full history of the loop. Every other field is last-write-wins (LangGraph's default).

6.2 The iteration-counting subtlety (read this before touching the loop)

This is the single most load-bearing, least intuitive detail in the whole codebase. `_route`

runs immediately after score_ats, **before** prepare_rewrite increments iteration:

```
def _route(self, state):
    passed = gate.passes(trust_reports[-1], ats_reports[-1])
    is_exhausted = state["iteration"] >= gate.max_iterations # pre-increment value
    return "end" if (passed or is_exhausted) else "rewrite"
```

With the default max_iterations=3:

iteration at _route	passed?	3 >= 3?	decision
0 (initial draft)	no	no	rewrite → iteration becomes 1
1	no	no	rewrite → 2
2	no	no	rewrite → 3
3	no	yes	end

Result: 4 total drafts (iterations 0, 1, 2, 3), not 3. Any change to the loop must keep this exact semantics — it's pinned by test_orchestrator.py::test_orchestrator_failsToCap_stopsAndExportsRealScores. Correspondingly, run() scales LangGraph's recursion limit as 6 + 4 * max_iterations + 10 rather than leaving LangGraph's default (25), so a caller-supplied higher cap doesn't hit a recursion-limit error before it hits its own cap.

6.3 WorkflowState — the stable public contract

```
class WorkflowState:
    user_id: str
    gate: QualityGate
    job_id: str | None # None for the legacy raw-posting path
    resume_id: str | None # set once TrustResumeApp._persist writes it
    job: JobDescription | None
    candidate_profile: CandidateProfile | None
    evidence: EvidenceSet | None
    drafts: list[ResumeDraft]
    trust_reports: list[TrustReport]
    ats_reports: list[ATSReport]
    iteration: int
    # properties: current_draft/current_trust/current_ats (last in each list),
    # passed (both scores meet threshold), is_exhausted (iteration >= cap),
    # should_continue
```

Orchestrator.run() builds GraphState, invokes the compiled graph via ainvoke, and converts the result dict back into WorkflowState on return — callers never see the graph shape.

6.4 Deterministic rewrite feedback (feedback.py)

build_feedback(trust, ats) -> str is plain string assembly, not another LLM call:

1. Hallucinated claims (removed/rephrase) — listed **first**, because accuracy outranks keyword coverage (the project's core premise).
2. Partially-supported claims (soften).
3. Missing keywords (incorporate, only where evidence genuinely supports them).
4. If none of the above fired (gate failed on aggregate score alone), a generic "aim for

Trust X, ATS Y" fallback.

6.5 CandidateProfileService — why "Service" not "Agent"

Wraps CandidateProfileAgent with **cache-check + document-assembly** logic that is itself deterministic (get_or_refresh): return the cached profile if stale=False; otherwise concatenate all the user's ingested chunks (ChunkRepository.list_for_user) into one blob, run the agent once, and cache the result keyed by a content hash (diagnostic only — staleness is decided purely by the stale flag, which IngestionService sets on any document mutation). Named "Service" — like IngestionService — because the orchestration logic around it is deterministic even though the thing it wraps makes an LLM call.

7. Ingestion (ingestion/service.py)

Not a pipeline step — a separate write path triggered by document upload/delete, independent of generate(). Its job is to land a document in both stores and keep them consistent, plus invalidate the candidate-profile cache on any mutation.

7.1 Pipeline

parse (unstructured, .docx/.pdf/.txt/.md) → clean_text (hand-rolled whitespace normalization) → dedup check → chunk_text (LangChain RecursiveCharacterTextSplitter) → write SQLite chunk rows → upsert Chroma vectors (roll back SQLite rows on failure)

7.2 Dedup: two-stage, at two layers

ingest_text(user_id, filename, text, document_type, job_id=None):

1. **Filename-first.** If this user already has a document with this exact filename, treat the upload as an update to that same logical document: - same cleaned-text hash → no-op (returns the existing id). - different hash → **reindex_in_place**: delete old chunks/vectors, re-chunk, re-embed, keep the same document_id (so job links/history survive). A Chroma failure here can't cleanly roll back to the *old* chunks (they're already gone) — an accepted, logged gap; there's no cross-store transaction anywhere in this codebase.
2. **Content-hash fallback**, for a never-before-seen filename: reuse the existing find_by_content_hash check — a re-upload of identical content under a new name is still recognized as a duplicate.

The DB also enforces UNIQUE(user_id, content_hash) as a second line of defense against the check-then-insert race (two concurrent identical ingests); ingest_new_document catches the resulting IntegrityError and re-reads the winning row rather than raising.

Every mutation (ingest_text success paths and delete_document) calls candidate_profiles.mark_stale(user_id) — delete_document's call runs in a finally so a mid-method failure can't leave the cache pointed at documents that no longer exist.

7.3 Parsing (parser.py)

One shared entry point for every non-plain-text format via unstructured.partition.auto.partition(..., strategy="fast"). "fast" is deliberate, not the default: unstructured's default "hi_res" loads a torch layout/OCR model — ~49s to parse one résumé PDF in testing — versus ~3s for "fast" (plain text extraction), for identical downstream content quality since this app never needs layout awareness.

7.4 Chunking (chunker.py)

chunk_text(text, *, max_chars=800, overlap=100) wraps LangChain's RecursiveCharacterTextSplitter with separators=["\n", " ", ""] — not the library default ["\n\n", ...], because clean_text already collapsed blank-line paragraph breaks to a single \n. Overlap is applied between every adjacent chunk pair (a real behavior change from the original's hand-rolled version, which only overlapped when hard-splitting an oversized paragraph) — the framework's default is the more standard RAG behavior, so chunk boundaries (and therefore retrieval hits and downstream scores)

differ from the original by design.

8. Trust & ATS scoring (pure, non-LLM logic)

8.1 Trust (trust_verification/verifier.py)

Pure functions with **zero LangChain import** (ADR-0004's separation of verification logic from the LLM wrapper): `format_draft`, `format_evidence`, `build_prompt`, `build_trust_report`. `TrustHarnessAgent` (the only thing that imports `langchain_core` in this area) calls these; the score itself (`TrustReport.compute_score`, §3) lives on the model, not here, since it's a data-level rubric the harness's classified claims feed into — separated so each layer is independently testable.

8.2 ATS (evaluation/scorer.py)

`score_keywords(draft, job) -> ATSReport`: flattens the draft to lowercased text, checks substring presence of each of `job.keywords` (falling back to `job.required_skills` if no keywords were extracted), scores `matched/total * 100`. **A job with no keywords scores 100** — nothing to fail against. `required_skill_coverage` is a separate, exact-token (non-substring) cross-check against a candidate's actual skill list, kept distinct because `required_skills` is written in full sentences (substring matching there would spuriously match, e.g., "SQL" inside "PostgreSQL").

9. API / application facade (api/)

9.1 TrustResumeApp (app_service.py) — where everything gets wired

Constructor takes injected collaborators only (a `sqlite3.Connection`, a Chroma client, an Embeddings, a `BaseChatModel`) — never constructs its own real dependencies, so tests build the exact same class with in-memory/fake versions and exercise the full stack with no HTTP, no network, no credentials. `build_default_app(db_path, chroma_path, llm_config)` is the one place that assembles the real thing (`FastEmbedEmbeddings`, `chromadb.PersistentClient`, `build_model(config)`).

Responsibilities, roughly grouped:

- **User/document management** — `add_document/add_document_bytes` (delegate to `IngestionService`), `list_documents`, `delete_document`.
- **Job management** — `create_job/update_job` extract+persist a `JobDescription` immediately (not lazily on first generation), sharing the *same* `JobDescriptionAgent` instance the orchestrator uses (so `Orchestrator._analyze_job` becomes a no-op when a persisted job is passed) — one source of truth, not two agents that happen to agree. `link_document_to_job`, `upload_document_for_job`, `list_documents_for_job` (generic pool + job-linked documents).
- **Ad-hoc retrieval** — `search_evidence(user_id, query, limit)` — the same `HybridRetriever` a generation uses internally, exposed standalone.
- **Generation** — `generate(user_id, job_posting)` (legacy, raw string) and `generate_for_job(user_id, job_id)` (reuses the persisted `JobDescription`, scopes retrieval to that job's eligible documents). Both call `Orchestrator.run` then `self._persist(state)`.
- **_persist** — writes the final draft + PDF/Markdown export bytes + Trust/ATS scores to `generated_resumes/evaluations`, unconditionally (even a capped-out, failing draft gets persisted with its real scores — ADR-0005). A failing draft additionally gets `rejection_reason` (`orchestration/rejection.py`'s `build_rejection_reason`) and `improvement_suggestions` (the same `build_feedback` output one more rewrite would have used).

9.2 model_factory.py — provider-agnostic LLM construction

`LLMConfig.load()` resolves provider config with precedence **env vars > config/llm.local.json (gitignored) > config/llm.json (committed) > hardcoded default**. provider ∈ {`bedrock`, `openai`, `google`, `test`}; each non-default provider's SDK is imported lazily inside its own branch of `build_model()`, so installing only the providers extra you actually need keeps the base install lean. Bedrock is the default and authenticates via an AWS profile (`aws_profile/aws_region`), not an API key.

9.3 The offline test provider (test_provider.py)

AutoStructuredFakeChatModel — LangChain's own GenericFakeChatModel doesn't implement `bind_tools`, so it can't drive with `structured_output` at all. This one does: it inspects whatever JSON Schema an agent's with `structured_output(Schema)` bound as a tool, and synthesizes a minimal valid instance of the schema's *required* fields only (empty string/list, first enum value, 0/0.0/False as appropriate), leaving Pydantic's own field defaults to fill in everything else. This is what makes `TRUSTRESUME_LLM_PROVIDER=test` able to drive the entire pipeline with zero credentials — pydantic-ai's `TestModel` had this "synthesize anything" ability natively; LangChain's fakes don't, so it's rebuilt here.

Consequence worth remembering: synthesized `_ClaimExtraction.claims` is always `[]`, so Trust score is always 0 under `provider=test` — offline mode always runs to the iteration cap rather than passing. Deterministic, not a bug.

9.4 FastAPI server (server.py)

`create_app(app_facade)` takes the facade by injection (what tests drive via `TestClient`); `build_served_app()` is the `--factory` uvicorn calls in production, which builds the *real* facade from environment variables only once the server actually starts (so importing the module has zero side effects).

Route surface (all against a single hardcoded `DEMO_USER_ID = "demo-user"` — every store call is still genuinely user-scoped underneath, so real auth slots in by replacing just that one constant):

Method + path	What it does
GET /api/health	liveness
GET /api/ping	live round-trip through the configured LLM provider (501 if SDK missing, 502 if the call fails)
GET/POST /api/documents, POST /api/documents/upload, DELETE /api/documents/{id}	document CRUD (JSON text or raw multipart upload, parsed server-side)
POST/GET /api/jobs, GET/PUT/DELETE /api/jobs/{id}	persisted job CRUD, extracting JobDescription at create/update time
POST /api/jobs/{id}/documents/upload, GET /api/jobs/{id}/documents	upload+link a document to a job in one step; list eligible documents
POST /api/search	standalone hybrid retrieval, no full generation
POST /api/generate	legacy raw-posting-string generation

Method + path	What it does
POST /api/jobs/{id}/generate	generation against a persisted job (no re-extraction, job-scoped documents)
GET /api/jobs/{id}/resumes, GET /api/resumes/{id}, GET /api/resumes/{id}/pdf, GET /api/resumes/{id}/markdown	browse/export past generations

Upload size is capped at 10 MiB (`_MAX_UPLOAD_BYTES`) — no legitimate résumé upload needs more, and this bounds per-request memory use rather than buffering an unbounded body.

Wire DTOs (`schemas.py`) are intentionally distinct from `models/` — e.g. `GenerateResponse.from_state(state: WorkflowState)` projects the internal state into a flatter shape (`draft`, `trust_score`, `ats_score`, `passed`, `exhausted`, `iterations`, `hallucinations: list[ClaimView]`, `missing_keywords`, `resume_id`) so the domain model can evolve without silently changing the public API contract.

10. Frontend (ui/)

Streamlit, and *only* a REST client: `ui/api_client.py`'s `TrustResumeClient` wraps requests. `Session` with one method per backend route; `ui/streamlit_app.py` renders four tabs (Documents, Generate, Jobs, Search) purely from those responses. No import from `trustresume.api` or any backend-internal module anywhere in this package — the dependency arrow is strictly UI → HTTP → backend, so the backend runs headless with no UI installed at all (streamlit/requests are the optional ui extra, not a core dependency).

`TRUSTRESUME_API_URL=http://localhost:8000 streamlit run src/trustresume/ui/streamlit_app.py`

A real bug caught only by actually running this (not by `ruff/mypy`): `streamlit run file.py` executes the file as a script with no package context, so a relative import (`from .api_client import ...`) raises `ImportError` — fixed by using the absolute `from trustresume.ui.api_client import TrustResumeClient`. Static analysis can't catch this because it doesn't know the difference between "imported as a module" and "run as a script."

11. Storage schema (`storage/schema.py`, SQLite)

SQLite Schema

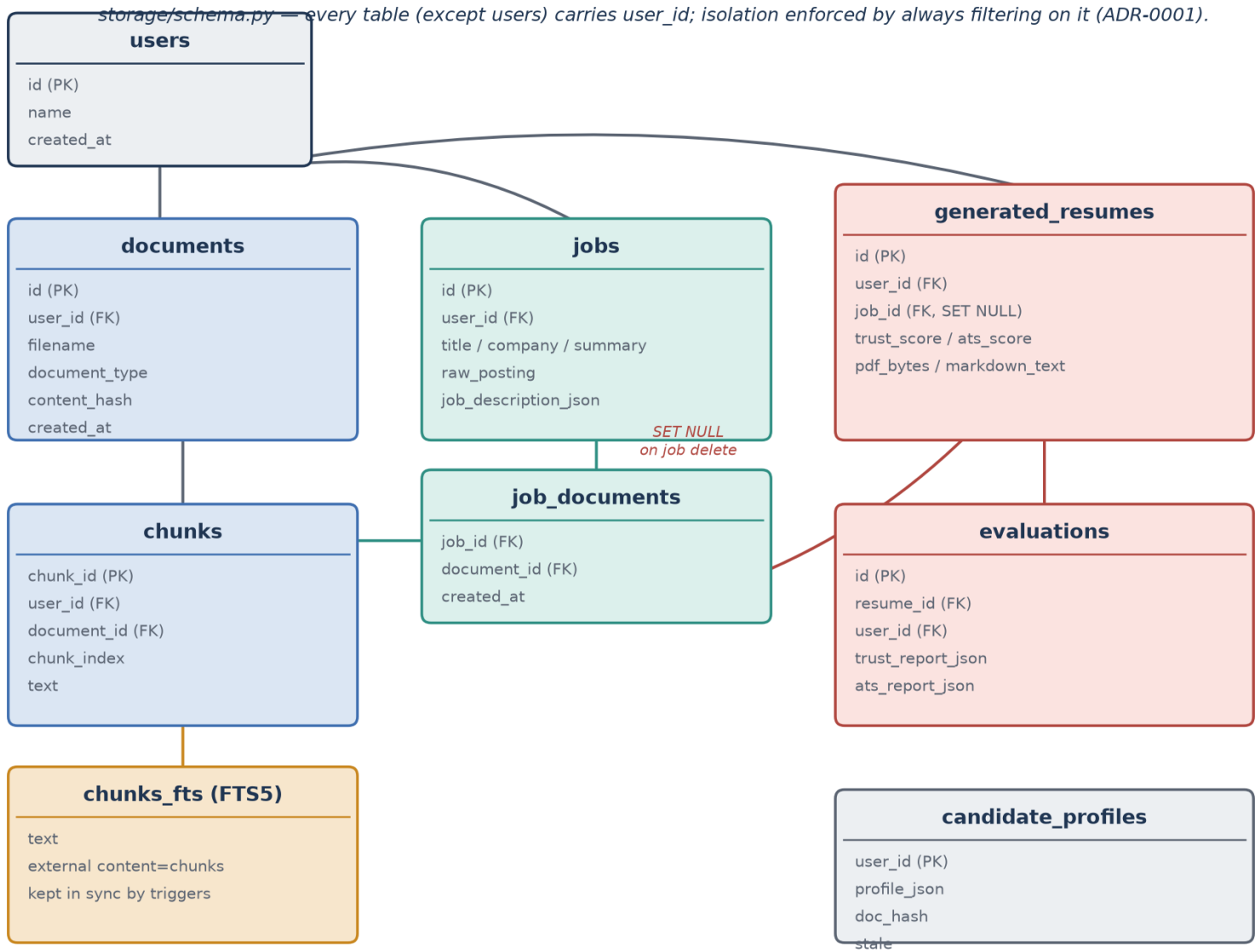


Figure 4. The SQLite schema — every table except users carries user_id.

```

users (id, name, created_at)
└ documents (id, user_id, filename, document_type, content_hash, created_at)
  UNIQUE(user_id, content_hash) -- dedup, DB-level backstop
  UNIQUE(user_id, filename) -- filename-first identity
└ chunks (chunk_id, user_id, document_id, chunk_index, document_type,
source document, text, created_at)
└ chunks_fts -- FTS5 external-content virtual table, kept in
sync by triggers on chunks; not a 3rd store
└ jobs (id, user_id, title, company, summary, raw_posting,
job_description_json, created_at, updated_at)
└ job_documents (job_id, document_id, created_at) -- M:N link table
└ generated_resumes (id, user_id, job_id [ON DELETE SET NULL], job_title,
iteration, summary, content_json, trust_score, ats_score, passed,
pdf_bytes, markdown_text, rejection_reason, improvement_suggestions,
created_at)
└ evaluations (id, resume_id, user_id, iteration, trust_score,
ats_score, trust_report_json, ats_report_json,
created_at)
└ candidate_profiles (user_id [PK], profile_json, doc_hash, stale,
updated_at)
  
```


Notable design choices:

- **jobs.job_id → generated_resumes is ON DELETE SET NULL, not CASCADE** — deleting a job must not destroy the historical record of résumés generated against it; job_title is flattened at persist time so the row stays meaningful even after its job is gone.
 - **Foreign keys are enforced per-connection** (PRAGMA foreign_keys = ON in connect()) — SQLite defaults this off.
 - **One shared connection, check_same_thread=False**, serves FastAPI's threadpool workers — safe because SQLite's default threading mode is serialized and every repository write commits immediately; revisit (connection pool / per-request connection) only if write concurrency grows.
 - **No migration tooling at all.** init_db only ever runs CREATE TABLE/INDEX IF NOT EXISTS — a schema change is invisible against a database file created before that change; the first write touching a new column fails at runtime, not at startup. Delete trustresume.db/ chroma_data locally, or docker compose down -v, before running code with schema changes against old data. This is an accepted, deliberate gap at this project's scale.
-

12. Observability (logging_config.py)

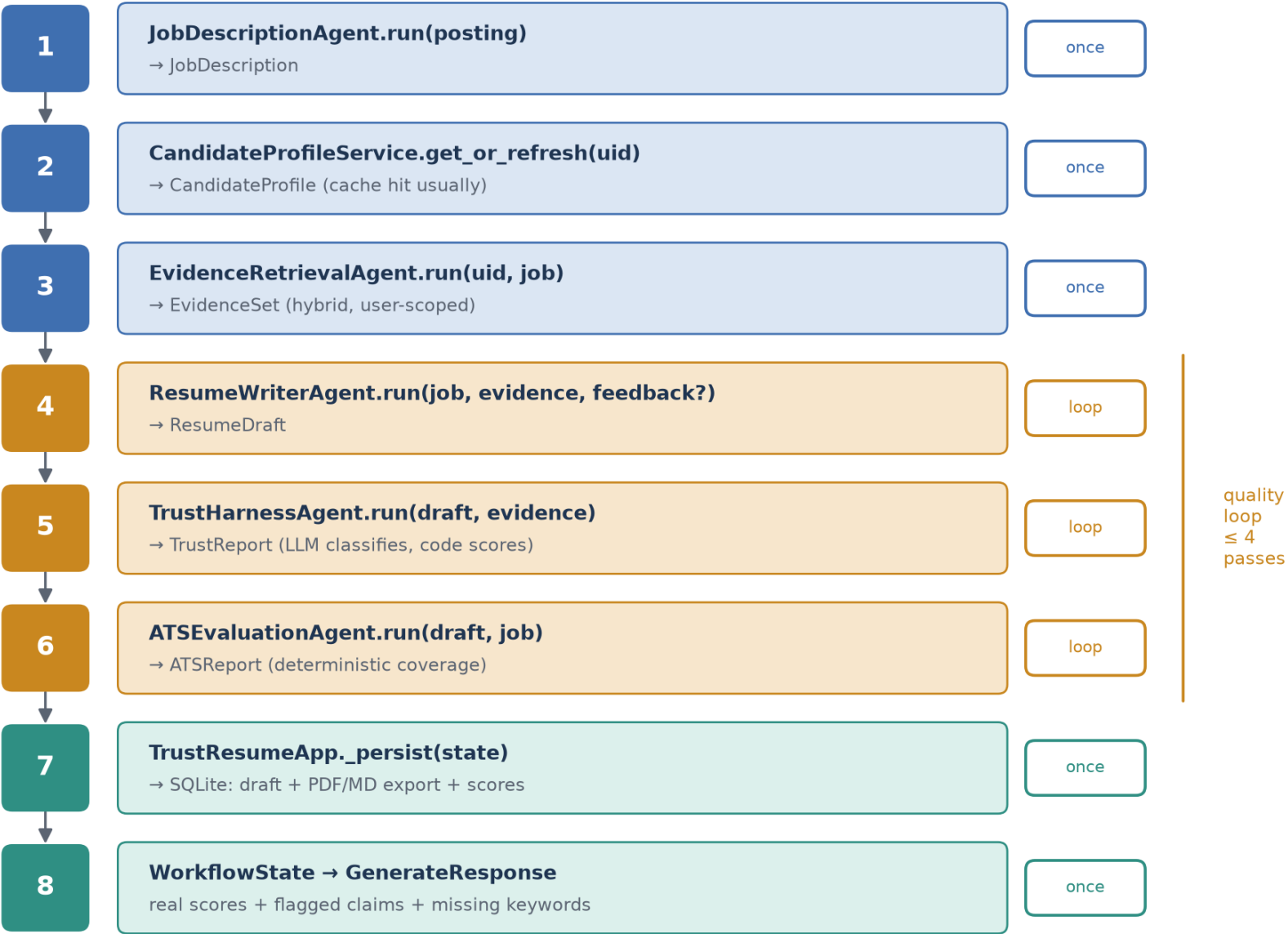
Stdlib logging (no third-party logging library) + a JsonFormatter that renders every record as one JSON object per line — the standard shape for container log collectors. configure_logging() is called exactly once, from server.py's build_served_app (the real process entry point) — never from library code, so importing any other module stays side-effect-free (matters for tests, which import freely without wanting stdout reconfigured).

The orchestrator logs every node transition and the quality-gate routing decision (_route's passed/is_exhausted/decision); ingestion logs success/rollback/dedup-skip.

One real bug worth knowing before adding your own extra={...}: extra={"filename": ...} (or module/name/lineno/etc.) collides with LogRecord's own reserved attribute names and raises KeyError **inside** stdlib logging — but only when a real logger call actually formats a record; a unit test that constructs LogRecord directly bypasses the check and won't catch it. This is why ingestion/service.py logs doc_filename, not filename. Regression test:
test_logging_config.py::test_loggerInfo_withExtra_doesNotRaiseOnReservedLookingKeys.

13. End-to-end data flow: one POST /api/generate call

One POST /api/generate — End-to-End Data Flow



Pass gate: Trust ≥ 90 AND ATS ≥ 85 → stop, PASS.
Otherwise, at iteration 3 already → stop, CAPPED (still persisted with real scores).
Otherwise → build_feedback(trust, ats) → back to step 4.

Figure 3. One POST /api/generate call, start to finish.

Assumes documents are already ingested.

- | | | |
|--|--------------------|-----------------------------|
| 1. JobDescriptionAgent.run(posting) | → JobDescription | (once) |
| 2. CandidateProfileService.get_or_refresh(uid) | → CandidateProfile | (once; usually a cache hit) |
| 3. EvidenceRetrievalAgent.run(uid, job) | → EvidenceSet | (once; hybrid, user-scoped) |


```

└── quality loop: ≤ 4 passes total (initial + up to 3 rewrites) ───┐
4. ResumeWriterAgent.run(job, evidence, feedback?) → ResumeDraft
5. TrustHarnessAgent.run(draft, evidence)           → TrustReport   (LLM classifies, code
scores)
6. ATSEvaluationAgent.run(draft, job)               → ATSReport     (deterministic coverage)
└── Trust ≥ 90 AND ATS ≥ 85 → PASS, stop
└── iteration == 3 already → CAPPED, stop (persist anyway, with real scores)
└── else → build_feedback(trust, ats) → prepare_rewrite (iteration += 1) → back to step 4
7. TrustResumeApp._persist(state): write final draft + PDF/Markdown export +
scores to SQLite; if failing, also rejection_reason + improvement_suggestions
8. WorkflowState → GenerateResponse (real scores, flagged hallucinations, missing keywords)

```

`generate_for_job(job_id)` is the same flow with two differences: step 1 is skipped (the persisted `JobDescription` is passed straight into the graph, so `_analyze_job` is a no-op), and step 3's retrieval is additionally scoped to that job's eligible document set (its generic, unlinked document pool plus anything explicitly linked to it).

14. Testing model

Whole stack runs offline — no network, no credentials (NFR-5).

Real dependency	Test double
SQLite file	<code>connect(":memory:")</code>
Chroma	<code>chromadb.EphemeralClient()</code> — unique collection name per test/app instance (its underlying storage is cached by settings hash, so same-named collections leak state across "fresh" clients within one process); <code>TrustResumeApp.__init__</code> 's <code>chroma_collection_name</code> param exists for exactly this
SQLite FTS5 keyword search	No fake — <code>:memory:</code> has FTS5 built in
Document parsing (.docx/.pdf)	<code>unstructured.partition.auto.partition</code> mocked for the element-joining unit test; two live-marked tests parse real sample files
Embedding model	<code>tests/fakes.py</code> 's <code>FakeEmbeddings</code> (SHA-256-hashed vectors); real <code>FastEmbedEmbeddings</code> 's lazy-load contract is mock-tested, its real output is live-marked

Real dependency	Test double
LLM (unit tests)	tests/fakes.py's <code>scripted_tool_call(name, args)</code> — a <code>FakeToolCallingChatModel</code> returning one scripted tool call; name must match the target schema's class name
LLM (provider=test, integration/live)	<code>AutoStructuredFakeChatModel</code> (§9.3)
Streamlit frontend	<code>streamlit.testing.v1.AppTest</code> drives the real script; only <code>requests.Session</code> is mocked at the <code>api_client</code> import site — no browser. <code>st.cache_resource.clear()</code> must run between tests (autouse fixture)

tests/unit/ mirrors src/trustresume/'s package boundaries 1:1; tests/integration/ exercises TrustResumeApp and the FastAPI app end-to-end; tests/integration/test_api_live.py boots a real uvicorn subprocess and is live-marked (deselected by default).

Coverage gate is enforced, not advisory: pytest's addopts bakes in `--cov-fail-under=95` on every invocation (actual ~99%) — a coverage regression fails the run itself. src/trustresume/poc/* is omitted (it's throwaway, credential-requiring smoke-test code by its own docstring).

```
pytest # offline unit + integration, coverage-gated
pytest -m live # + real embedding model / real uvicorn
subprocess
pytest tests/unit/test_foo.py::test_bar # one test
pytest -k "expression" # tests matching an expression
ruff check . # lint
ruff format --check src tests # format check
mypy src # strict type-check
```

15. Deployment

Deployment — Docker Compose

(same multi-stage image; `runtime` and `ui` differ only in CMD)

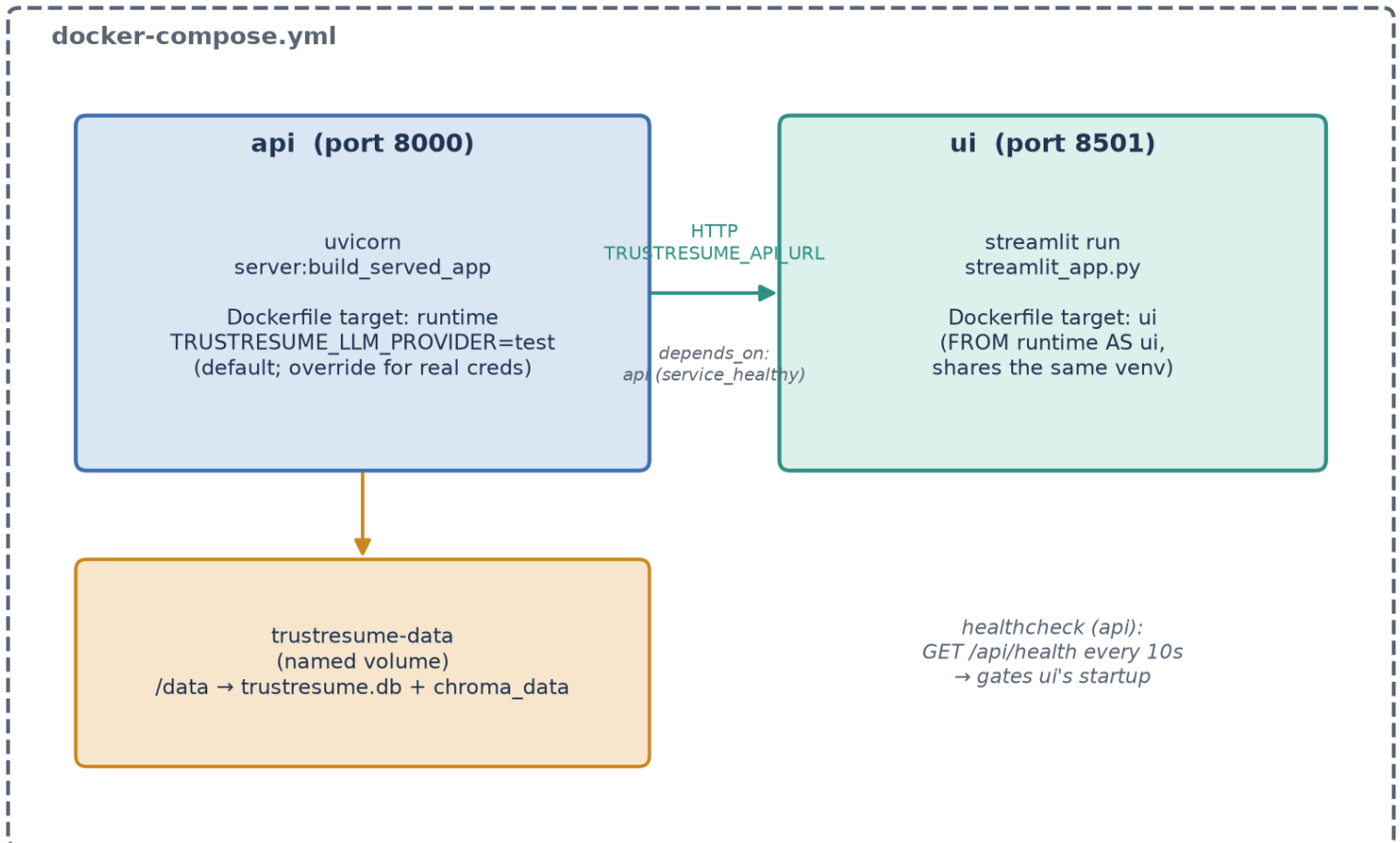


Figure 5. Docker Compose topology — one image, two services.

15.1 Local, no credentials

```
uv venv --python 3.13 && source .venv/bin/activate
uv sync --locked --extra dev --extra providers --extra ui
```

```
TRUSTRESUME_LLM_PROVIDER=test uvicorn trustresume.api.server:build_served_app --factory
--port 8000
# in another shell:
TRUSTRESUME_API_URL=http://localhost:8000 streamlit run src/trustresume/ui/streamlit_app.py
```

15.2 Docker Compose

Two services sharing one image (Dockerfile, multi-stage):

- **builder** — `uv sync --locked --extra providers --extra ui` into one venv (cache-friendly: dependency install layer is separated from the COPY src layer, since deps change far less often).
- **runtime** — `slim python:3.13-slim + libgomp1` (fastembed's ONNX runtime needs it), non-root user, CMD `uvicorn ... --factory`. Env vars bake in `TRUSTRESUME_DB_PATH=/data/trustresume.db`, `TRUSTRESUME_CHROMA_PATH=/data/chroma_data`; /data is a declared VOLUME.

- **ui** — FROM runtime AS ui, same venv/source, different CMD (streamlit run ...); talks to the API stage over HTTP (TRUSTRESUME_API_URL=http://api:8000), so it needs no volume of its own.

docker-compose.yml wires api (port 8000, trustresume-data volume, healthcheck against /api/health) + ui (port 8501, depends_on: api condition: service_healthy). TRUSTRESUME_LLM_PROVIDER defaults to test in compose, so docker compose up --build needs zero credentials out of the box; override it (plus the matching credentials) for a real provider.

```
docker compose up --build
```

Note: per §11, there's no schema migration path — docker compose down -v to drop trustresume-data before starting on top of a volume created by an older schema version.

15.3 CI (.github/workflows/ci.yml)

On every push to main and every PR, across Python 3.11 and 3.13:

```
uv sync --locked --extra dev --extra providers --extra ui
→ ruff check .
→ ruff format --check src tests
→ mypy src
→ pytest (coverage gate from pyproject.toml, --cov-fail-under=95)
```

Dependencies always install from the committed uv.lock (--locked, never re-resolved) — the whole point being that local, CI, and Docker builds land on byte-identical dependency versions.

15.4 Configuration surface (all optional, env-first)

Variable	Meaning	Default
TRUSTRESUME_LLM_PROVIDER (or legacy TRUSTRESUME_LLM)	bedrock / openai / google / test	bedrock
TRUSTRESUME_LLM_MODEL	model id/name	provider default
TRUSTRESUME_AWS_PROFILE / TRUSTRESUME_AWS_REGION	Bedrock only	twdc-bedrock-central / us-west-2
OPENAI_API_KEY / GOOGLE_API_KEY	read by the provider SDK directly	—
TRUSTRESUME_DB_PATH	SQLite file path	trustresume.db
TRUSTRESUME_CHROMA_PATH	Chroma storage dir	chroma_data

Variable	Meaning	Default
TRUSTRESUME_API_URL	(UI only) backend base URL	http://localhost:8000

Resolution precedence for provider/model/key fields specifically: **env var** > **config/llm.local.json (gitignored)** > **config/llm.json (committed defaults)** > **hardcoded default** — see `LLMConfig.load()` (§9.2).

16. Where to go deeper

- Exact rationale for a specific decision → `architecture/decisions/000{1,3}.md`, `0010-hybrid-vector-keyword-retrieval.md`.
- A guided, narrative first read of the whole codebase (written in Chinese) → `docs/code-walkthrough.md`.
- Command reference, package map table, testing-model table → `CLAUDE.md`.
- What was and wasn't ported from the original pydantic-ai/Qdrant project, and how to pull further upstream changes → `SYNC.md`.